

Taller 03 Posix Semáforos e Hilos

Marlon García

Carrera de Ingeniería de Sistemas

Departamento de Ingeniería de Sistemas

Pontificia Universidad Javeriana

John Jairo Corredor Franco PhD.

19 de Mayo del 2026

Índice

Resumen	3
1. Introducción	4
2. Refactorización de la Solución Multihilo (posixSincro.c)	5
2.1. El Problema de la Finalización Violenta	5
2.2. Sincronización Elegante	5
3. Reingeniería del Sistema Multiproceso (IPC)	6
3.1. Soporte Modular y Prevención de Fugas	6
4. Búsqueda Concurrente de Máximos (concurrencyPosix.c)	7
4.1. Lógica de Partición y Distribución	7
4.2. Evaluación Masiva de Pruebas	7
5. Automatización de Compilación y Organización (Makefile)	8
6. Conclusiones	9
7. Referencias	10

Resumen

Este informe detalla la evolución técnica y metodológica de un sistema clásico de sincronización y procesamiento paralelo utilizando la API POSIX en C. El proyecto aborda dos paradigmas fundamentales de concurrencia: el modelo Productor-Consumidor y el paralelismo de datos (Data Parallelism). Mediante una reingeniería sistemática, se transformó un software heredado en un sistema robusto, modularizado en un directorio `src/` y tolerante a fallos. Se analiza el paso de una concurrencia multihilo (`posixSincro.c`) a una arquitectura multiproceso real coordinada mediante variables en memoria compartida (`shm`) y semáforos POSIX (`producer.c`, `consumer.c`). Adicionalmente, se integra y documenta un nuevo programa (`concurrenciaPosix.c`) dedicado a la partición de grandes volúmenes de datos para la búsqueda concurrente del valor máximo utilizando `pthread`s. Toda la solución está orquestada de forma limpia por un archivo de automatización de compilación (`Makefile`). Los resultados demuestran la viabilidad de lograr robustez y escalabilidad a nivel industrial en código de bajo nivel en sistemas UNIX.

1 Introducción

La programación concurrente y de sistemas es uno de los pilares del diseño de software moderno. Sin embargo, trae consigo desafíos inherentes como condiciones de carrera (*race conditions*), bloqueos mutuos (*deadlocks*) y la gestión del estado compartido.

En este laboratorio se abordan dos patrones arquitectónicos críticos:

- **Sincronización por Intercambio de Tareas (Productor-Consumidor):** Se implementa tanto de manera Multihilo (compartiendo el espacio de direccionamiento virtual) como Multiproceso (con espacios aislados comunicados vía Memoria Compartida y Semáforos POSIX).
- **Paralelismo de Datos (Búsqueda Distribuida):** Se emplea un esquema *Scatter-Gather* (dispersión y recolección) sobre arreglos masivos de información para hallar eficientemente un valor máximo delegando el cálculo a múltiples hilos de forma simultánea.

El presente documento relata el proceso de refactorización y extensión de la base de código para solucionar problemas críticos encontrados originalmente, modularizar la solución y validar el paralelismo con arreglos de hasta 1.000.000 de enteros.

2 Refactorización de la Solución Multihilo (`posixSincro.c`)

El programa `posixSincro.c` planteaba un modelo multihilo clásico con 10 productores que escribían en un búfer circular de tamaño 5 y un hilo “spooler” (consumidor/impresor) que extraía los datos para imprimirlos.

2.1 El Problema de la Finalización Violenta

En la versión heredada, el hilo principal esperaba de manera ineficiente mediante un ciclo de espera activa (*busy wait*) basado en `sleep`, utilizando posteriormente `pthread_cancel` para abortar bruscamente al hilo impresor. Esta estrategia es peligrosa y propensa a generar *deadlocks* y fugas de memoria.

2.2 Sincronización Elegante

Para solucionar esto, se introdujo una variable de estado global protegida llamada `active_producers` que realiza un seguimiento del número de hilos productores vivos. El flujo de finalización ahora se diseña de manera que el hilo `spooler` detecte de forma cooperativa cuándo debe terminar su ejecución.

```
1 // C digo refactorizado (posixSincro.c)
2 while (!lines_to_print && active_producers > 0)
3     pthread_cond_wait(&spool_cond, &buf_mutex);
4
5 if (!lines_to_print && active_producers == 0) {
6     pthread_mutex_unlock(&buf_mutex);
7     break; // Salida elegante del bucle de impresi n
8 }
```

Listing 1: Condición de salida ordenada en el Spooler

3 Reingeniería del Sistema Multiproceso (IPC)

Los verdaderos desafíos de reingeniería residieron en la comunicación multiproceso (`producer.c` y `consumer.c`). El código original carecía de modularidad e impedía una inicialización robusta.

3.1 Soporte Modular y Prevención de Fugas

La primera medida correctora consistió en diseñar una interfaz común (`consumer.h`) para centralizar la estructura de la memoria de intercambio (`shm_open`) y la declaración de semáforos nombrados (`sem_open`). Además, se añadió una bandera `terminado` para flexibilizar el flujo de datos.

Para resolver la fragilidad en la puesta en marcha, se implementó un mecanismo de reintentos dinámico (Bucle Paciente) en el consumidor. Si al arrancar el consumidor los recursos aún no han sido instanciados por el productor, el sistema entra en un bucle controlado de espera (hasta 15 intentos) en lugar de abortar abruptamente.

Finalmente, para prevenir la acumulación de archivos huérfanos en `/dev/shm` al abortar el programa (`Ctrl+C`), se instaló un capturador de señales del sistema operativo (`SIGINT` y `SIGTERM`) en ambos ejecutables:

```
1 void limpiar_recursos(int sig) {  
2     (void)sig;  
3     printf("\n[Sistema] Interrumpido. Limpiando IPC...\n");  
4     sem_unlink("/vacio");  
5     sem_unlink("/lleno");  
6     shm_unlink("/memoria_compartida");  
7     exit(EXIT_SUCCESS);  
8 }
```

Listing 2: Manejador de señales para desvincular recursos IPC

4 Búsqueda Concurrente de Máximos (`concurrencyPosix.c`)

Como parte de la evolución del proyecto, se introdujo y completó un nuevo programa diseñado para procesar archivos de texto masivos y hallar el valor numérico máximo utilizando **paralelismo de datos**.

4.1 Lógica de Partición y Distribución

El programa `concurrencyPosix.c` toma como entrada un archivo de números y la cantidad de hilos a crear (N). El tamaño total del arreglo se divide uniformemente entre la cantidad de hilos.

```
1 // Calculo del rango del arreglo para cada hilo
2 int elementos_por_hilo = n / nhilos;
3 int resto = n % nhilos;
4 int inicio = 0;
5
6 for(int i = 0; i < nhilos; i++){
7     args[i].inicio = inicio;
8     args[i].fin = inicio + elementos_por_hilo + (i < resto ? 1 : 0);
9     args[i].vector = vec;
10    args[i].maxparcial = INT_MIN; // Iniciar con minimo absoluto
11    // ... Creación de pthread ...
12    inicio = args[i].fin;
13 }
```

Listing 3: División dinámica de carga entre hilos

Cada hilo procesa su segmento de memoria de manera aislada y guarda su resultado en un atributo de salida `maxparcial`. Luego, el hilo principal (*main thread*) realiza un bloque `pthread_join` en bucle para recolectar las respuestas y calcular el máximo global.

4.2 Evaluación Masiva de Pruebas

Para certificar la fiabilidad y rapidez del programa frente a cargas industriales, se generó la suite de pruebas automatizada (`pruebas/run_tests.sh`) que evalúa el algoritmo con archivos autogenerados extremos:

- Un archivo mediano de **10.000 elementos**.
- Un archivo grande de **100.000 elementos**.
- Un archivo de estrés (`test3_negativos.txt`) con **1.000.000 de enteros puramente negativos** y un límite máximo inyectado de `-5`. El programa procesa satisfactoriamente estos flujos inicializando su variable de comparación en `INT_MIN`.

5 Automatización de Compilación y Organización (Makefile)

Toda la base de código (los cuatro ejecutables principales y las cabeceras) fue migrada a la estructura de carpeta limpia `src/`. Para compilar este ecosistema, se amplió el Makefile asegurando que los ejecutables resultantes queden expuestos directamente en el directorio raíz.

El archivo define reglas genéricas y enlaza explícitamente el soporte de hilos POSIX (`-pthread`) y la biblioteca de extensiones en tiempo real de POSIX (`-lrt`):

```
1 CC = gcc
2 CFLAGS = -Wall -Wextra -pthread
3 LDFLAGS = -pthread -lrt
4
5 TARGETS = posixSincro producer consumer concurrenciaPosix
6
7 all: $(TARGETS)
8
9 posixSincro: src/posixSincro.c
10     $(CC) $(CFLAGS) $< -o $@
11
12 producer: src/producer.c src/consumer.h
13     $(CC) $(CFLAGS) $< -o $@ $(LDFLAGS)
14
15 consumer: src/consumer.c src/consumer.h
16     $(CC) $(CFLAGS) $< -o $@ $(LDFLAGS)
17
18 concurrenciaPosix: src/concurrenciaPosix.c
19     $(CC) $(CFLAGS) $< -o $@
```

Listing 4: Makefile automatizado para el entorno modular

6 Conclusiones

1. **Gestión de Memoria y Señales de Kernel:** En el IPC multiproceso, el ciclo de vida de los recursos sobrepasa la vida del proceso que los creó. Un interceptor de señales cooperativo para limpiar la memoria compartida previene la degradación silente del sistema operativo Linux anfitrión.
2. **Eficacia del Paralelismo Distribuido:** La implementación en `concurrencyPosix.c` demostró la viabilidad matemática de fracturar un *array* de 1 millón de elementos en subarreglos continuos y analizarlos en hilos independientes sin bloqueos (usando el patrón *Scatter-Gather*), lo que maximiza la capacidad de un CPU moderno y evita latencias de `mutex` al no requerir sincronización fina.
3. **La Sincronización Cooperativa Multihilo:** Se ratifica que un proceso multihilo no debe usar jamás la cancelación agresiva. Intercambiar la función `pthread_cancel` por condiciones lógicas atómicas (`active_producers`) erradicó las fugas de hilos de forma definitiva en la solución Productor-Consumidor clásica.

7 Referencias

Referencias

- [1] Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4ta ed.). Pearson Education.
- [2] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10ma ed.). John Wiley & Sons.